AutoRF

Overview

AutoRF is the first agentic AI framework for automated radio frequency (RF) hardware design, supporting both metasurfaces and antennas. Given a user's design requirements expressed in natural language, AutoRF automatically translates them into structured hardware specifications, retrieves matching design templates, proposes initial circuit models, and iteratively optimizes design parameters through electromagnetic (EM) simulation — producing validated RF hardware designs with minimal human intervention.

AutoRF is released as a research artifact accompanying our paper. This repository contains the AutoRF framework, including the Demand Compiler, Design Proposer, and Design Maker modules, along with the supporting design library and simulation tool interfaces.

What can AutoRF do

AutoRF automates the end-to-end RF hardware design process, which traditionally requires extensive domain expertise and manual iteration with simulation tools. Specifically, AutoRF:

- Translates natural-language design requirements into structured hardware specifications using a set of specification primitives (frequency band, polarization, operation mode, substrate material, layer configuration, and more)
- Searches a curated design library of parameterized templates to retrieve the best-matching design and proposes an initial circuit model
- Performs fine-grained electromagnetic optimization using the HFSS simulator with iterative LLM-driven parameter tuning, automatically adjusting design variables to meet target performance
- Supports a wide design space including metasurfaces operating from 900 MHz to 160 GHz, multiple substrate materials (FR4, paper, Rogers, airgap), single and multi-layer configurations, phase and amplitude control, transmissive and reflective surfaces, and both programmable (active) and passive designs
- Supports antenna design with impedance matching at target frequencies
- Achieves over 10× speedup compared to gradient-based variable optimization in HFSS, while requiring only ~10 design templates compared to the ~100K training samples needed by prior approaches

A detailed discussion of AutoRF's design, implementation, and evaluation can be found in our paper at:

Intended uses

AutoRF is intended for research and experimental use by RF engineers, antenna designers, and electromagnetic researchers who:

- Study automated approaches to RF hardware design and optimization
- Develop or evaluate AI-assisted design workflows for metasurfaces, antennas, and related RF components
- Wish to reproduce or extend the results presented in the AutoRF paper
- Explore the capabilities of LLM-driven agents in domain-specific hardware design tasks

AutoRF is being shared with the research community to facilitate reproduction of our results and foster further research in AI-assisted RF hardware design.

AutoRF is intended to be used by domain experts in RF engineering who are independently capable of evaluating the quality of generated designs — including simulation results, circuit models, and fabrication parameters — before acting on them.

Out-of-scope uses

AutoRF is not intended for:

- Direct use in production manufacturing pipelines without independent verification by qualified RF engineers
- Safety-critical RF system design (e.g., medical devices, aviation radar, emergency communication systems) without thorough additional validation
- Use by non-experts who lack the RF domain knowledge to evaluate the correctness and physical realizability of generated designs
- Autonomous design and fabrication without human review of simulation results and physical constraints

We do not recommend using AutoRF in commercial or real-world applications without further testing and development. It is being released for research purposes.

AutoRF was not designed or evaluated for all possible downstream purposes. Developers should consider its inherent limitations as they select use cases, and evaluate and mitigate for accuracy, safety, and fairness concerns specific to each intended downstream use.

Without further testing and development, AutoRF should not be used in sensitive domains where inaccurate outputs could suggest actions that lead to injury or negatively impact an individual's legal, financial, or life opportunities.

We do not recommend using AutoRF in the context of high-risk decision making (e.g. in law enforcement, legal, finance, or healthcare).

How to get started

To begin using AutoRF, clone the repository and follow the setup instructions in the README provided in the root of the project's repository.

AutoRF requires access to:

- **An LLM API** — AutoRF uses large language models for demand compilation, design proposal, and optimization. Configure your preferred LLM endpoint (e.g., Azure OpenAI) as described in the repository documentation.
- **Ansys HFSS** — The Design Maker module interfaces with the HFSS electromagnetic simulator for fine-grained EM optimization. A valid HFSS installation and

license are required for full functionality.

- **Python environment** — AutoRF includes a Python-based circuit model simulator for coarse-grained probing and a custom API abstraction layer wrapping HFSS APIs.

A typical workflow is: (1) install dependencies and configure LLM and HFSS access; (2) provide a natural-language design requirement (e.g., "Design a reflective metasurface operating at 28 GHz on an FR4 substrate"); (3) AutoRF compiles the demand, proposes a design, and iteratively optimizes it; (4) review the generated design parameters and simulation results.

Evaluation

AutoRF was evaluated on its ability to automatically design RF hardware (metasurfaces and antennas) from natural-language user requirements, measuring both end-to-end design success rate and physical validation through fabrication.

A detailed evaluation methodology and results are provided in the accompanying paper at:

Evaluation methods

We evaluated AutoRF using:

- **End-to-end success rate** on 100 metasurface user demand samples spanning the full design space (900 MHz to 160 GHz, multiple substrate types, single/multi-layer, active/passive, transmissive/reflective)
- **Antenna design success rate** across a set of antenna design tasks with impedance matching requirements
- **Physical fabrication and measurement** of 7 metasurface designs and 2 antenna designs ranging from 2.4 GHz to sub-THz frequencies, comparing simulated performance against real-world measurements
- **Design speed comparison** against HFSS's built-in gradient-based variable optimizer to quantify speedup
- **Comparison against prior automated approaches** including Karahan et al. and MetaGen, which require ~100K training designs for narrower design spaces

AutoRF uses large language models for its core modules (Demand Compiler, Design Proposer, and Design Maker). The framework is model-agnostic — results may vary depending on the specific LLM used, its capabilities, and configuration. The evaluation employed rejection sampling fine-tuning with filtered trajectories to improve reliability, alongside a rule reviewer agent that provides automated feedback.

Evaluation results

AutoRF achieved a 92% end-to-end design success rate on 100 metasurface user demand samples and a 93% success rate for antenna design tasks. Across case studies, AutoRF demonstrated over 10× speedup compared to the gradient-based variable optimizer in HFSS.

Physical validation through fabrication of 7 metasurface designs and 2 antennas (spanning 2.4 GHz to sub-THz frequencies) showed strong agreement between simulated and measured performance, confirming the practical viability of AutoRF-generated designs.

Compared to prior automated approaches, AutoRF covers a significantly wider design space — supporting 900 MHz to 160 GHz, multi-layer configurations, active and passive surfaces, and both transmissive and reflective designs — while requiring only ~10 parameterized templates, versus the ~100K training designs needed by methods such as MetaGen (which is limited to 60–80 GHz passive single-layer metasurfaces).

Limitations

AutoRF was developed for research and experimental purposes. Further testing and validation are needed before considering its application in commercial or real-world scenarios.

AutoRF has the following specific limitations:

- **Simulation tool dependency**: AutoRF's Design Maker module relies on Ansys HFSS for fine-grained electromagnetic simulation. Users must have a valid HFSS installation and license. The framework's optimization quality is bounded by the accuracy and capabilities of the underlying simulator.
- **Design library coverage**: AutoRF's Design Proposer retrieves from a curated library of parameterized templates. Designs that fall significantly outside the existing template library may not be well-served. Expanding the template library to cover additional RF component types requires manual curation by domain experts.
- **LLM-generated outputs require expert review**: The Demand Compiler, Design Proposer, and Design Maker modules all rely on LLM-generated outputs — including hardware specification parsing, circuit model proposals, and parameter optimization decisions. These outputs may contain errors, physically unrealizable configurations, or suboptimal parameter choices. All designs should be validated by qualified RF engineers before fabrication.
- **Optimization convergence**: The iterative LLM-driven parameter tuning in the Design Maker may not converge for all design requirements, particularly for highly constrained or unusual specifications. The 92% success rate indicates that approximately 8% of metasurface demands did not produce satisfactory designs.
- **Fabrication gap**: While simulation results showed strong agreement with measurements, some deviation is expected due to manufacturing tolerances and material variations not captured in simulation. Physical prototyping and measurement remain necessary validation steps.

AutoRF inherits any biases, errors, or omissions produced by the LLM it is used with. Developers are advised to choose an appropriate base LLM carefully, depending on the intended use case.

AutoRF's fine-tuned models inherit any biases, errors, or omissions characteristic of their training data, which may be amplified by AI-generated interpretations of design requirements or simulation results.

There has not been a systematic effort to ensure that systems using AutoRF are protected from security vulnerabilities such as indirect prompt injection attacks. Since AutoRF processes natural-language design requirements as input to its LLM-based modules, any deployment should take proactive measures to validate and sanitize inputs as appropriate.

Best practices

To achieve the best results with AutoRF, we recommend:

- **Start with well-defined requirements**: Provide clear, specific design requirements including target frequency, substrate material, number of layers, and operation mode. Ambiguous or underspecified demands may lead to suboptimal designs.
- **Validate simulation results before fabrication**: Always review the EM simulation results produced by the Design Maker and verify that the design meets all physical constraints before proceeding to fabrication.
- **Use the circuit model simulator for rapid screening**: Leverage the Python-based circuit model simulator for coarse-grained design exploration before committing to full EM simulation in HFSS, which is more computationally expensive.
- **Verify LLM-parsed specifications**: Review the structured specifications output by the Demand Compiler to ensure that the natural-language requirements were correctly translated into hardware parameters before proceeding to design proposal and optimization.
- **Expand the design library incrementally**: When working with RF components not well-represented in the existing template library, consider adding new parameterized templates to improve coverage and retrieval quality.

We strongly encourage users to use LLMs/MLLMs that support robust Responsible AI mitigations, such as Azure Open AI (AOAI) services. Such services continually update their safety and RAI mitigations with the latest industry standards for responsible use. For more on AOAI's best practices when employing foundation models for scripts and applications:

- What is Azure AI Content Safety?
- Overview of Responsible AI practices for Azure OpenAI models
- Azure OpenAI Transparency Note
- OpenAI's Usage policies
- Azure OpenAI's Code of Conduct

Developers should follow transparency best practices and inform end-users they are interacting with an AI system. Users should be made aware that AutoRF-generated designs are AI-assisted and require expert validation before fabrication or deployment.

License

MIT license

Nothing disclosed here, including the Out of Scope Uses section, should be interpreted as or deemed a restriction or modification to the license the code is released under.

Trademarks

This project may contain trademarks or logos for projects, products, or services. Authorized use of Microsoft trademarks or logos is subject to and must follow Microsoft's Trademark & Brand Guidelines. Use of Microsoft trademarks or logos in modified versions of this project must not cause confusion or imply Microsoft sponsorship. Any use of third-party trademarks or logos are subject to those third-party's policies.

Contact

This research was conducted by members of Microsoft Research Asia. We welcome feedback and collaboration from our audience. If you have suggestions, questions, or observe unexpected/offensive behavior in our technology, please contact us at:

- Ruichun Ma —
- Lili Qiu —
- Hao Pan —

If the team receives reports of undesired behavior or identifies issues independently, we will update this repository with appropriate mitigations.